

CYBERSECURITY

LAB 7

I. Introduction.

The application of cryptographic methods is very wide. The laboratory focuses on aspects related to digital signatures and Public Key Infrastructure (PKI). Digital signatures are commonly used daily, so in a security context, it is important to become familiar with their operation and properties. A public-key infrastructure (PKI) consists of protocols, standards, and services that establish and support the applications of such a trust system.

Digital signature is mainly used for authentication purposes. It is used to convince communicating parties with each other's identity and exchange their session keys. It is an electronic format of signature that can be used by a person to authenticate the identity of the message's sender or identity of the document's signer. It ensures that the original content of the message or document sent is intact. In other words, it is a cryptographic tool for signing messages, data, and verifying the authenticity of the message, data. Digital signatures use a standard, accepted format, called Public Key Infrastructure (PKI), to provide the highest levels of security and universal acceptance. Moreover, digital signatures are widely used today in the business, e.g. for authorizing bank payments, for exchange of signed electronic documents, for signing transactions in the public blockchain systems, for signing digital contracts, and in many other scenarios. The digital signature provides:

- **authentication** - it allows checking if the sender created and signed the message,
- **integrity** - it allows you to check that the data has not been altered after the signature,
- **non-repudiation** - the person signing the message cannot deny his signature,

The digital signature process consists of three algorithms:

- **key generation** - this generates a set of keys, private and public,
- **signing** - the algorithm signs the message with a private key, usually the message hash is encrypted.
- **verification** - the algorithm checks the authenticity of the message by verifying the signature with a public key,

Asymmetric cryptographic methods are used to sign messages/data. The digital signature process prevents several common attacks such as:

- *key-only* – Attacker has access to the public key
- *known message* – Attacker has access to valid signatures for known messages, but not those that they have chosen
- *adaptive chosen message* – Attacker gains access to signatures on various messages that they have chosen.

The most popular public-key cryptosystems are:

- RSA,
- DSA (Digital Signature Algorithm),
- ECDSA / EdDSA based on elliptic curves (ECC),

LAB 7

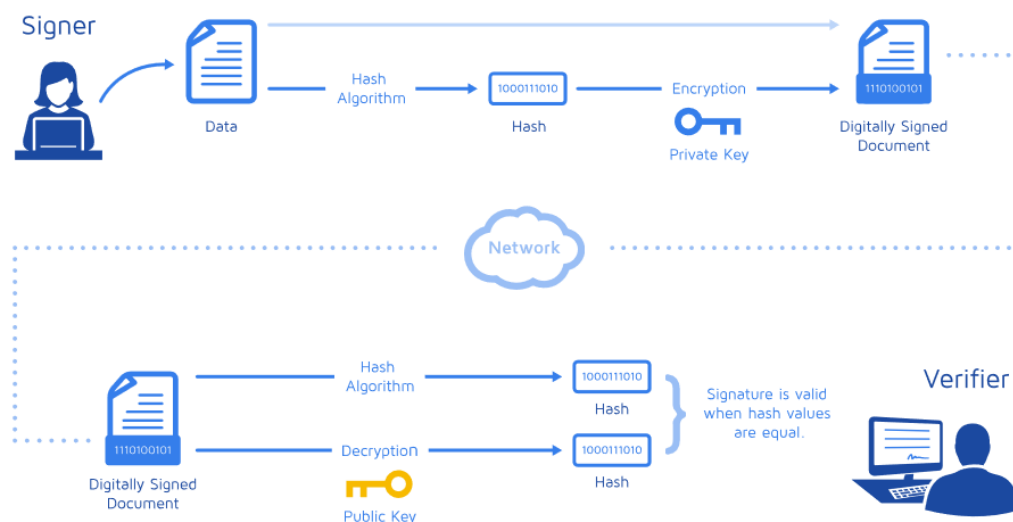


Figure 1 Flow of the digital signatures - source <https://www.docusign.com/how-it-works/electronic-signature/digital-signature/digital-signature-faq>

Public key infrastructure (PKI) allows efficient and secure identification of public keys. It can be used within or between organizations with the help of the Internet. Different types of PKI can be deployed by varying the essential configuration details, trust rules. PKI is a set of identities, roles, policies, and actions for the creation, use, management, distribution, and revocation of public and private keys. There are several PKI standards in the industry, the most popular is the X.509 version 3. Moreover, PKI provides:

- authentication,
- confidentiality,
- non-repudiation,
- integrity,

PKI management model involves specifying the rules for message formats and procedures used to communicate. The model also explains how various entities communicate with each other. The major entities of PKI management are as follows:

End Entity (EE): It can be a user or software application to which the certificate is served. It needs secure access at least to its name and private key.

Certification Authority (CA): It may be a third party or from the EE's organization that issues a certificate to the EE.

Registration Authority (RA): It is a subset of EE and is an optional component. If RA is not present, then CA performs RA's functions. RA carries out functions such as key generation, keypair management, token distribution, etc.

Certificate Revocation Lists issuer (CRL): If some certificates have to be revoked, the CRL issuer will take care of it. It is also an optional component.

Certificate Repository: Storage unit to define how to store certificates and CRLs and how it can be accessed by the EE.

The Certificate Signing Request (CSR) should be generated on the client's computer. CSR contains the public key and ID to be certified. The CSR is then sent to the RA/CA to sign the certificate. Afterward, the certificate becomes fully valuable. Generating CSR on the customer's computer is important because only then the private key is known to the owner, and other entities do not have access to it.

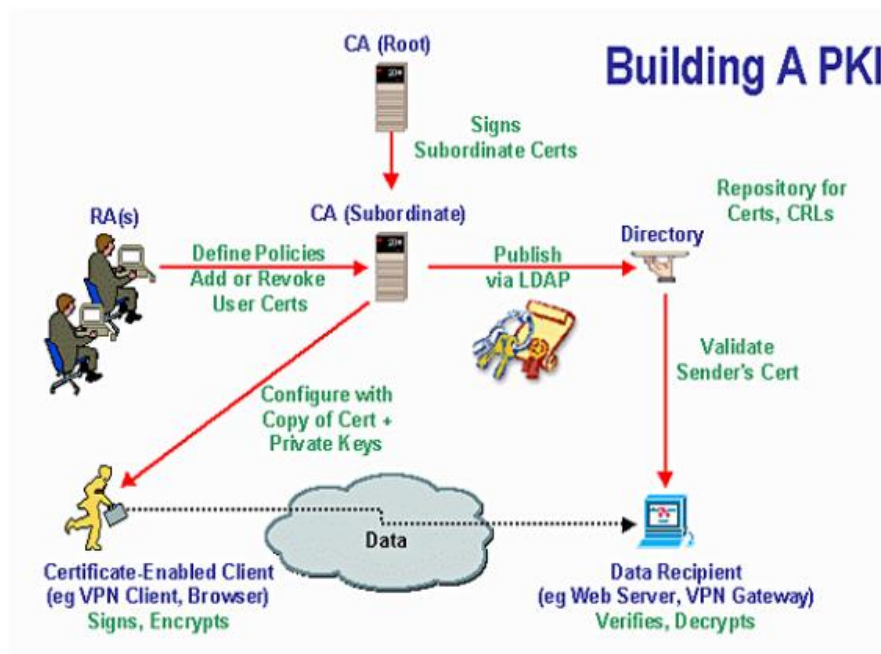


Figure 2 PKI concepts - source <https://cybersecuritynews.co.uk/how-does-public-key-infrastructure-pki-work>

II. Application of modern cryptography.

In this part of the task, a set of keys should be generated, which are necessary for further tasks. In the first part, a set of keys is to be generated using OpenSSL (RSA algorithm), in the second part, the OpenPGP tool is used.

OpenSSL:

Basic commands:

Use the OpenSSL tool to generate RSA private key (key length 2048 bits):

```
openssl genrsa -out private_key.pem 2048
```

Based on private key, generate public.key:

```
openssl rsa -in private_key.pem -outform PEM -pubout -out public_key.pem
```

Sign the file with the following commands:

```
openssl dgst -sha256 -sign private_key.pem -out encrypted_hash.sha256 data.txt
```

To verify the signature, run the following command:

```
openssl dgst -sha256 -verify public_key.pem -signature encrypted_hash data.txt
```

OpenSSL documentation is available here: <https://www.openssl.org/docs/>

OpenPGP:

Basic commands:

Generate key pair using GPG tool:

```
gpg --expert --full-gen-key
```

Before generating the keys, you are obligated to complete the following fields: appropriate algorithm, properties of an algorithm (e.g. key length, elliptic curve, etc.), expiry date, name, e-mail address, and passphrase.

To list available keys use:

LAB 7

- public:

```
gpg --list-keys,
```

- private:

```
gpg --list-secret-keys,
```

Export a public key:

- `gpg --output --export <file name> --armour <email_address>` - text form - ASCII,
- `gpg --export --output <file name> <email_address>` - binary form,

Export a private key:

- `gpg --export-secret-keys -a <file name> <email address>`

Importuj klucz:

- `gpg --import <file name>`

Sign a file:

- `gpg --sign <file name>,`

Sign a file with a selected key:

- `gpg --sign -u <key_id> <file name>,`

Verify a signature:

- `gpg --verify <file name>.gpg`

Sign a imported key:

- `gpg --sign-key`

Encrypt a file with gpg:

- `gpg --encrypt --recipient <email address> --output <output file> <input file>`

Other useful operations of gpg:

- `gpg --fingerprint`
- `gpg --edit-key`

1. Tasks:

- 1.1. Generate a set of keys using OpenSSL (RSA) and OpenPGP (ECC and RSA).
- 1.2. Export a public key from GPG (ASCII format).
- 1.3. Transfer your public key (OpenSSL and OpenPGP) to another machine/user account.
- 1.4. For OpenPGP, a public key should be imported and signed with a local private key. Verify a fingerprint of a certificate on both machines/ user accounts.
- 1.5. Create a text file and sign it using both methods.
- 1.6. Transfer your file and signatures to another machine/user account and verify the signatures.
- 1.7. After the signature has been successfully verified, modify the content of the original document and verify the digital signature again.

LAB 7

- 1.8. Only for OpenPGP. Encrypt the file using a public key and then decrypt the encrypted file. Afterward, change one character in the encrypted file and try to decrypt it again.

Questions:

- 1.9. Is it possible to generate only one of the key pairs for asymmetric algorithms (e.g. private key)? Would this make sense?
- 1.10. What form does the GPG key have? How does a PEM key differ from an OpenPGP key?
- 1.11. Is the exchange of private keys justified?
- 1.12. What is the fingerprint?
- 1.13. Describe what has changed and why in task 7?
- 1.14. Did you succeed in changing one character in an encrypted file and decrypt the encrypted text? Justify your answer. (Task 8)
- 1.15. What is the difference in the signature (OpenPGP) for ECC and RSA algorithms?
- 1.16. Can a message be signed with a public key? If so, what could the consequences be?

III. PKI - key generation in X.509 standard.

OpenVPN utilizes certificates to encrypt traffic between the server and clients. The simple certificate authority (CA) is built to issue trusted certificates. The Easy-RSA is a tool that is used to build and manage a PKI CA. Easy-RSA is based on the OpenSSL library.

2. Build CA.

The EasyRSA tool is located in home directory (/home/server). The tool is used to build and manage OpenVPN certificates:

Generation the sets of certificates:

Navigate to the EasyRSA directory:

```
cd ~/EasyRSA-3.0.10/
```

Copy the vars.example file:

```
cp vars.example vars
```

Edit the vars file:

```
nano vars
```

Remove # to uncomment lines and adjust values as follows:

```
#set_var EASYRSA_REQ_COUNTRY    "PL"
#set_var EASYRSA_REQ_PROVINCE   "Wroclaw"
#set_var EASYRSA_REQ_CITY       "Wroclaw"
#set_var EASYRSA_REQ_ORG        "PWR"
#set_var EASYRSA_REQ_EMAIL      "<student e-mail>"
#set_var EASYRSA_REQ_OU         "PWR"
```

Run this script with the `init-pki` option to initiate the public key infrastructure on the CA server:

```
./easyrsa init-pki
```

LAB 7

To build CA, run the following command - it generates ca.key (private.key) and ca.crt (public certificate). It is very important to keep the key in a safe place because the security of the VPN network depends on it.

```
./easyrsa build-ca nopass
```

Note that the common name should be correctly set according to its intended use.

To revoke a client certificate, you need to revoke the client on CA:

```
./easyrsa revoke <client's common name>
```

Then generate a CRL file and transfer it onto the server:

```
./easyrsa gen-crl
```

3. Generate the server and client certificates.

Run the command to generate a server private key (server.key) and a certificate signing request - CSR (server.req):

```
./easyrsa gen-req server nopass
```

Sign a server request using CA, it generates server.crt:

```
./easyrsa sign-req server server
```

Run the command to generate a client private key (client1.key) and a certificate signing request (client1.req):

```
./easyrsa gen-req client1 nopass
```

Sign a client signing request using CA, it generates client1.crt:

```
./easyrsa sign-req client client1
```

Generate Diffie-Hellman key (dh.pem) which is used to exchange keys:

```
./easyrsa gen-dh
```

Generate an HMAC signature (ta.key) to strengthen the server's TLS integrity verification capabilities:

```
openvpn --genkey --secret ta.key
```

The server files:

- server.key - server private key,
- server.crt - server certificate,
- ta.key HMAC signature,
- dh.pem - Diffie-Hellman key,
- ca.crt - CA certificate,

The client files:

- client.key - server private key,
- client.crt - server certificate,
- ta.key - HMAC signature,
- ca.crt - CA certificate,

Check the properties of one of the certificates and answer the following questions:

3.1.What is the signature algorithm used?

3.2.What is the public key algorithm used?

3.3.What version of the X.509 standard has been used to generate a certificate?

3.4.What else can be read from the server/client certificate?